Lab 9 Report

ECSE-4770 Computer Hardware Design

Sangeeth Thayaaparan

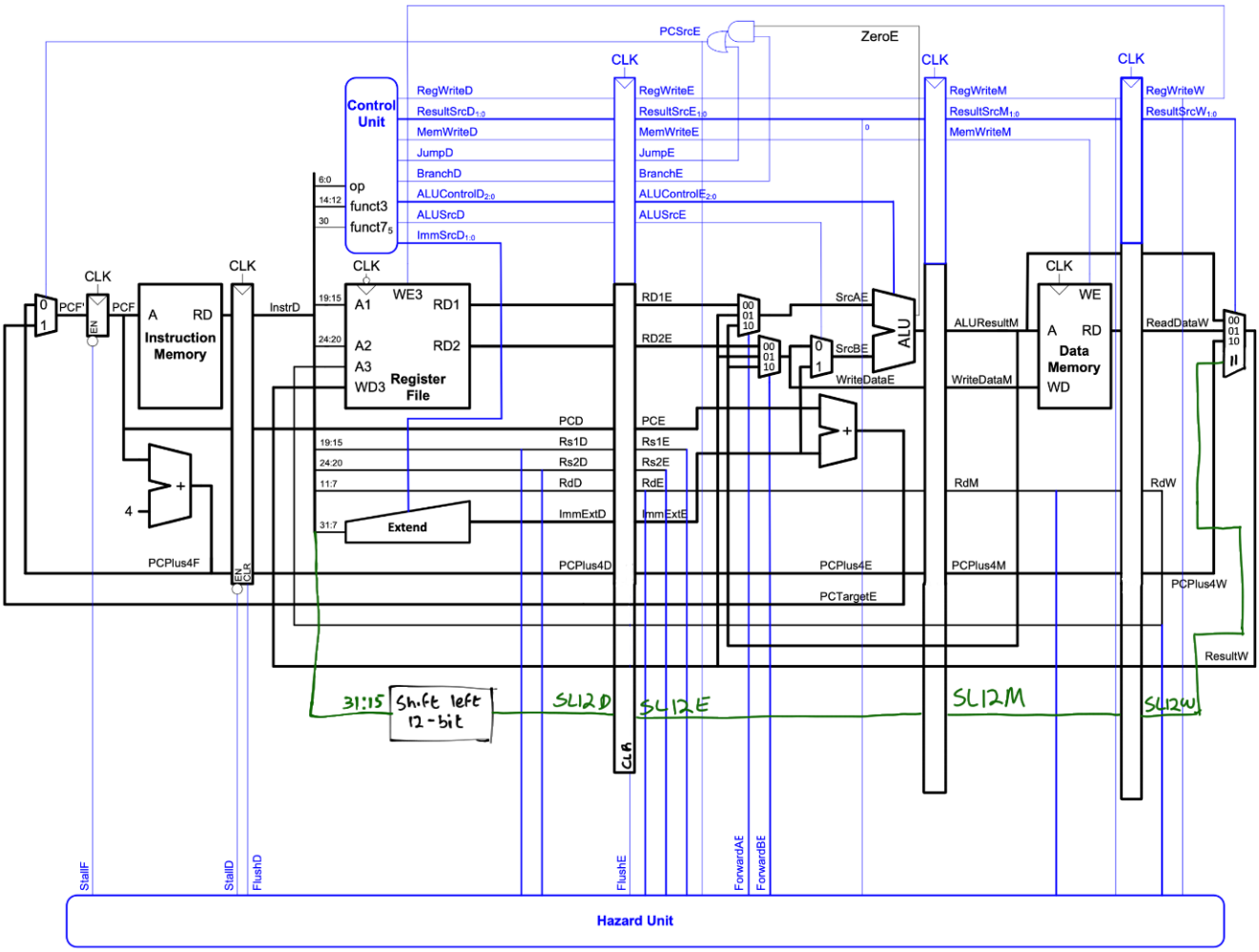
Time Taken

This lab took 6-8 hours to do. A lot of the time I spent was debugging various parts of the design, connecting different wires to different modules took some time.

Another portion that took a good amount of time was creating a testbench program that didn't have hazards. I found a tool online that converted assembly to machine code which significantly helped with this process:

- https://github.com/Kritagya-Agarwal/Assembly-To-Machine-Code-RISC-V
- https://luplab.gitlab.io/rvcodecjs/
    - Another tool I used for verifying individual lines instead of compiling an entire assembly program

Figure 1 Markup



LUI Supported Tables

Table 1. Main Decoder Truth Table

Instruction	Opcode	RegWrite	ImmSrc	ALUSrcA	ALUSrcB	MemWrite	ResultSrc	Branch	ALUOp	Jump

Instruction	Opcode	RegWrite	ImmSrc	ALUSrcA	ALUSrcB	MemWrite	ResultSrc	Branch	ALUOp	Jump
lw	0000011	1	00	0	1	0	01	0	00	0
sw	0100011	0	01	0	1	1	xx	0	00	0
R-type	0110011	1	xx	0	0	0	00	0	10	0
beq	1100011	0	10	0	0	0	xx	1	01	0
I-type ALU	0010011	1	00	0	1	0	00	0	10	0
jal	1101111	1	11	x	x	0	10	0	xx	1
lui	0110111	1	11	x	x	0	11	0	xx	0

Table 2. ALU Decoder Truth Table

This table remains unchanged

ALUOp[1:0]	funct3[2:0]	{op[5], funct7[5]}	ALUControl[2:0]	Operation
00	x	x	000	Add
01	x	x	001	Subtract
10	000	00, 01, 10	000	Add
10	000	11	001	Subtract
10	010	x	101	SLT
10	110	x	011	OR
10	111	x	010	AND

Testbench for LUI

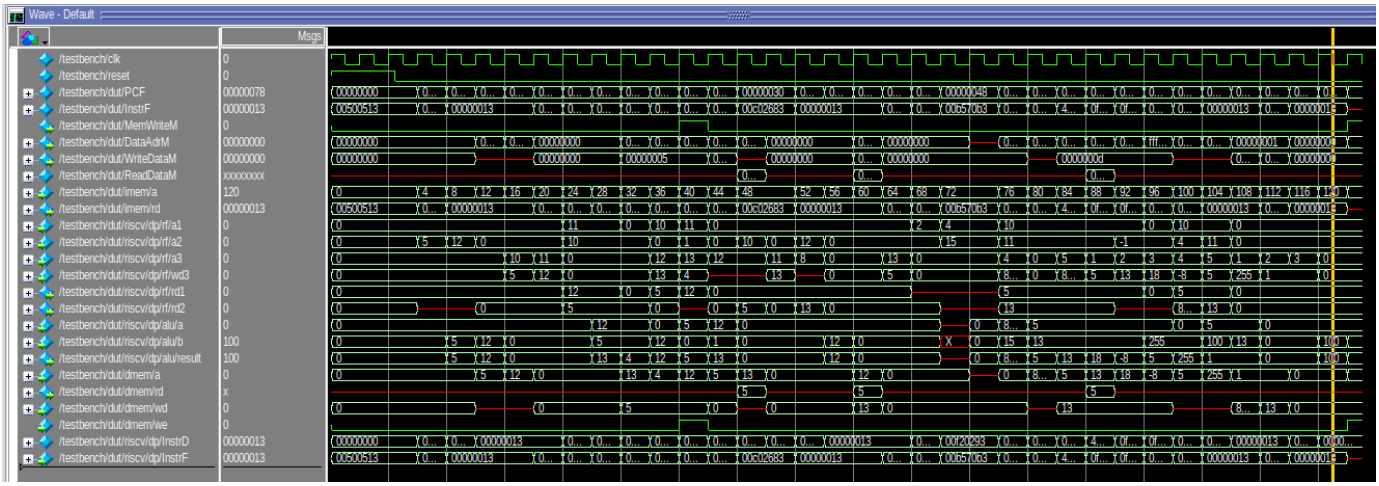
```
main:
#       Instruction             Description             Address         Machine Code
        addi x10, x0,  5        # x10 = 5               0x0             0x00500513
        addi x11, x0,  12       # x11 = 12              0x4             0x00C00593
        addi x0,  x0,  0        # nop                   0x8             0x00000013
        addi x0,  x0,  0        # nop                   0xC             0x00000013
        addi x0,  x0,  0        # nop                   0x10            0x00000013
        or   x12, x11, x10      # x12 = 17              0x14            0x00A5E633
        and  x13, x11, x10      # x13 = 4               0x18            0x00A5F6B3
        sw   x10, 12(x0)        # Mem[12] = 5           0x1C            0x00A02623
        beq  x10, x0, skip      # not taken             0x20            0x00050663
        addi x11, x11, 1        # x11 = 13               0x24            0x00158593
        beq x0, x0, taken       # taken                 0x28            0x00000463

skip:   addi x3, x0, 10         # shouldn't happen      0x2C            0x00A00193
taken:  lw   x13, 12(x0)        # x13 = Mem[12]         0x30            0x00C02683
        addi x0,  x0,  0        # nop                   0x34            0x00000013
        addi x0,  x0,  0        # nop                   0x38            0x00000013
        addi x0,  x0,  0        # nop                   0x3C            0x00000013

        lui x4, 20              # x4 = 20 * 2^12        0x40            0x00014237
        # gets forwarded
        addi x5, x4, 15         # x5 = x4 + 15          0x44            0x00F20293
        # x10 = 5 = 0x0101, x11 = 13 = 0x1101
        and x1, x10, x11        # x1 = x10 & x11        0x48            0x00B570B3
        or x2, x10, x11         # x2 = x10 | x11        0x4C            0x00B56133
        add x3 , x10, x11       # x3 = x10 + x11        0x50            0x00B501B3
```

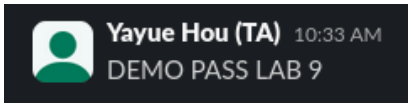
```
        sub x4, x10, x11        # x4 = x10 - x11        0x54            0x40B50233
        andi x5, x10, 255       # x5 = 5                0x58            0x0FF07293
        ori x1, x0, 255         # x1 = 255              0x5C            0x0FF06093
        slti x2, x10, 100       # x2 = 1                0x60            0x06452113
        slt x3, x10, x11        # x3 = 0                0x64            0x00B521B3
        addi x0,  x0,  0        # nop                   0x68            0x00000013
        addi x0,  x0,  0        # nop                   0x6C            0x00000013
        sw   x0,  100(x0)       # Mem[100] = 0          0x70            0x06002223
        # Program should end here
        addi x0,  x0,  0        # nop                   0x74            0x00000013
        addi x0,  x0,  0        # nop                   0x78            0x00000013
```

Simulation Waveforms



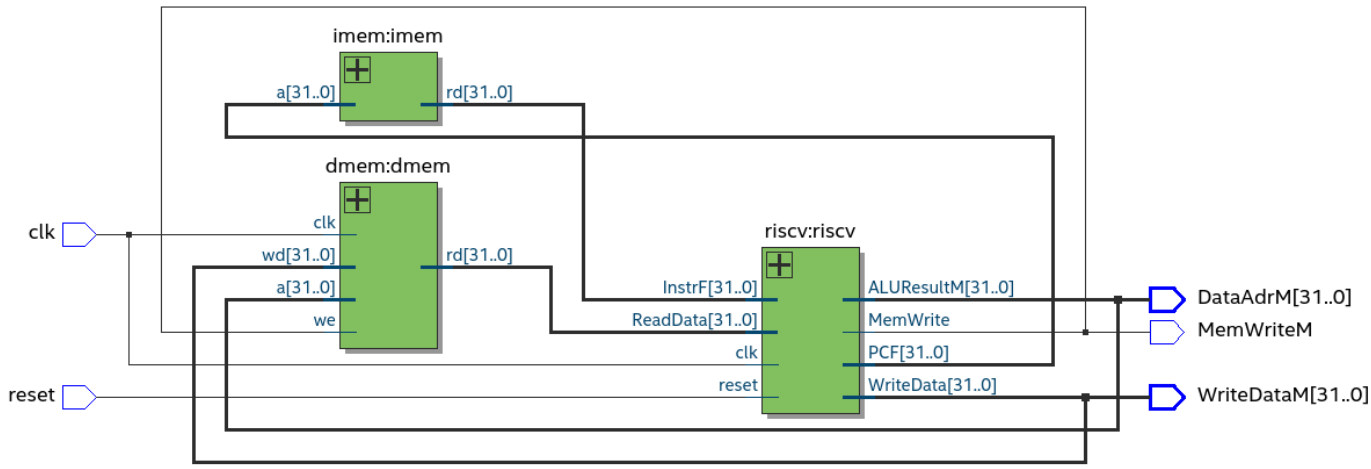
This contains additional waves that I used for debugging, but the required waves are located at the top.

TA Lab Checkoff

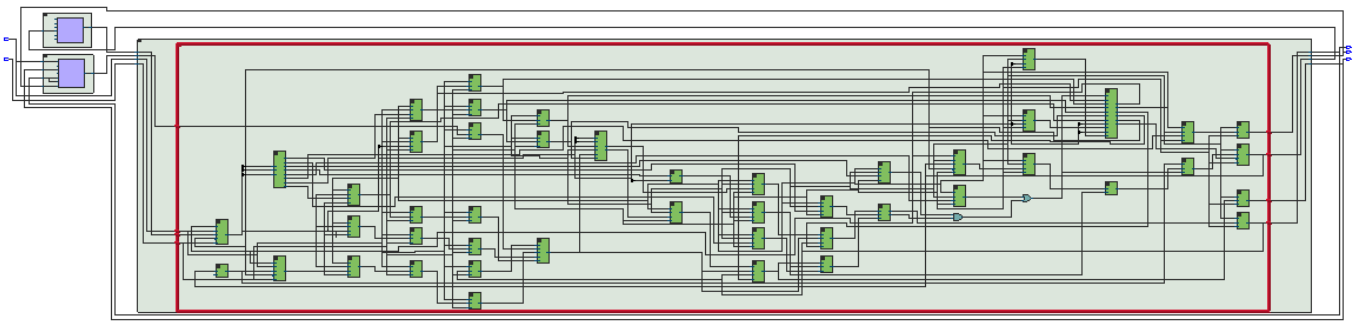


RTL Viewer Schematic

Condensed RTL view of the processor



Expanded, much less readable



Hierarchical SystemVerilog

```
// riscvsingle.sv

// RISC-V single-cycle processor
// From Section 7.6 of Digital Design & Computer Architecture
// 27 April 2020
// David_Harris@hmc.edu
// Sarah.Harris@unlv.edu

// run 210
// Expect simulator to print "Simulation succeeded"
// when the value 25 (0x19) is written to address 100 (0x64)

// Single-cycle implementation of RISC-V (RV32I)
// User-level Instruction Set Architecture V2.2 (May 7, 2017)
// Implements a subset of the base integer instructions:
//    lw, sw
//    add, sub, and, or, slt,
//    addi, andi, ori, slti
//    beq
//    jal
// Exceptions, traps, and interrupts not implemented
// little-endian memory

// 31 32-bit registers x1-x31, x0 hardwired to 0
// R-Type instructions
//   add, sub, and, or, slt
//   INSTR rd, rs1, rs2
//   Instr[31:25] = funct7 (funct7b5 & opb5 = 1 for sub, 0 for others)
//   Instr[24:20] = rs2
//   Instr[19:15] = rs1
//   Instr[14:12] = funct3
//   Instr[11:7]  = rd
//   Instr[6:0]   = opcode
// I-Type Instructions
//   lw, I-type ALU (addi, andi, ori, slti)
//   lw:         INSTR rd, imm(rs1)
//   I-type ALU: INSTR rd, rs1, imm (12-bit signed)
//   Instr[31:20] = imm[11:0]
//   Instr[24:20] = rs2
//   Instr[19:15] = rs1
//   Instr[14:12] = funct3
//   Instr[11:7]  = rd
//   Instr[6:0]   = opcode
// S-Type Instruction
//   sw rs2, imm(rs1) (store rs2 into address specified by rs1 + immm)
//   Instr[31:25] = imm[11:5] (offset[11:5])
//   Instr[24:20] = rs2 (src)
//   Instr[19:15] = rs1 (base)
```

```
//   Instr[14:12] = funct3
//   Instr[11:7]  = imm[4:0]  (offset[4:0])
//   Instr[6:0]   = opcode
// B-Type Instruction
//   beq rs1, rs2, imm (PCTarget = PC + (signed imm x 2))
//   Instr[31:25] = imm[12], imm[10:5]
//   Instr[24:20] = rs2
//   Instr[19:15] = rs1
//   Instr[14:12] = funct3
//   Instr[11:7]  = imm[4:1], imm[11]
//   Instr[6:0]   = opcode
// J-Type Instruction
//   jal rd, imm  (signed imm is multiplied by 2 and added to PC, rd = PC+4)
//   Instr[31:12] = imm[20], imm[10:1], imm[11], imm[19:12]
//   Instr[11:7]  = rd
//   Instr[6:0]   = opcode

//   Instruction  opcode    funct3    funct7
//   add          0110011   000       0000000
//   sub          0110011   000       0100000
//   and          0110011   111       0000000
//   or           0110011   110       0000000
//   slt          0110011   010       0000000
//   addi         0010011   000       immediate
//   andi         0010011   111       immediate
//   ori          0010011   110       immediate
//   slti         0010011   010       immediate
//   beq          1100011   000       immediate
//   lw           0000011   010       immediate
//   sw           0100011   010       immediate
//   jal          1101111   immediate immediate

module testbench();

  logic        clk;
  logic        reset;

  logic [31:0] WriteData, DataAdr;
  logic        MemWrite;

  // instantiate device to be tested
  top dut(clk, reset, WriteData, DataAdr, MemWrite);

  // initialize test
  initial
    begin
      reset <= 1; # 22; reset <= 0;
    end

  // generate clock to sequence tests
  always
    begin
      clk <= 1; # 5; clk <= 0; # 5;
    end

  // check results
  always @(negedge clk)
    begin
      // if(MemWrite) begin
      //   if(DataAdr === 100 & WriteData === 25) begin
      //     $display("Simulation succeeded");
      //     $stop;
      //   end else if (DataAdr !== 96) begin
      //     $display("Simulation failed");
```

```
      //     $stop;
      //   end
      // end
      if(DataAdr === 32'h00000064) begin
        $display("Simulation reached end");
        $stop;
      end
    end
endmodule

module top(input logic clk, reset,
           output logic [31:0] WriteDataM, DataAdrM,
           output logic MemWriteM);

  logic [31:0] PCF, InstrF, ReadDataM;

  // instantiate processor and memories
  riscv riscv(clk, reset, PCF, InstrF, MemWriteM, DataAdrM,
              WriteDataM, ReadDataM);

  imem imem(PCF, InstrF);
  dmem dmem(clk, MemWriteM, DataAdrM, WriteDataM, ReadDataM);
endmodule

module riscv(input  logic        clk, reset,
             output logic [31:0] PCF,
             input  logic [31:0] InstrF,
             output logic        MemWrite,
             output logic [31:0] ALUResultM,
             output logic [31:0] WriteData,
             input  logic [31:0] ReadData);

  logic        PCSrcE, RegWriteW;

  // Control signals from controller (Decode stage)
  logic       RegWriteD, MemWriteD, BranchD, JumpD;
  logic [1:0] ImmSrcD, ResultSrcD;
  logic       ALUSrcD;
  logic [2:0] ALUControlD;

  // Signals passed from Decode to Execute
  logic       RegWriteE, MemWriteE, BranchE, JumpE;
  logic [1:0] ImmSrcE, ResultSrcE;
  logic       ALUSrcE;
  logic [2:0] ALUControlE;

  // instantiate datapath (contains controller instantiation internally)
  datapath dp(clk, reset,
              InstrF,
              PCF, ALUResultM, WriteData, ReadData,
              MemWrite);
endmodule

module controller(input  logic [6:0] op,
                  input  logic [2:0] funct3,
                  input  logic       funct7b5,
                  output logic       RegWrite,
                  output logic [1:0] ResultSrc,
                  output logic       MemWrite,
                  output logic       Jump,
                  output logic       Branch,
                  output logic [2:0] ALUControl,
                  output logic       ALUSrc,
                  output logic [1:0] ImmSrc);
```

```
  // Main Decoder Truth Table
  // Instruction  Opcode    RegWrite ImmSrc ALUSrcA ALUSrcB MemWrite ResultSrc Branch ALUOp
Jump
  // lw           0000011   1        00     0       1       0        01        0      00
0
  // sw           0100011   0        01     0       1       1        xx        0      00
0
  // R-type       0110011   1        xx     0       0       0        00        0      10
0
  // beq          1100011   0        10     0       0       0        xx        1      01
0
  // I-type ALU   0010011   1        00     0       1       0        00        0      10
0
  // jal          1101111   1        11     x       x       0        10        0      xx
1
  // lui          0110111   1        11     x       x       0        11        0      xx
0

  logic [1:0] ALUOp_internal;
  logic RtypeSub;

  // Main Decoder
  always_comb
    case(op)
      7'b0000011: begin // lw
        RegWrite = 1'b1;
        ImmSrc = 2'b00;
        ALUSrc = 1'b1;
        MemWrite = 1'b0;
        ResultSrc = 2'b01;
        Branch = 1'b0;
        ALUOp_internal = 2'b00;
        Jump = 1'b0;
      end
      7'b0100011: begin // sw
        RegWrite = 1'b0;
        ImmSrc = 2'b01;
        ALUSrc = 1'b1;
        MemWrite = 1'b1;
        ResultSrc = 2'bxx;
        Branch = 1'b0;
        ALUOp_internal = 2'b00;
        Jump = 1'b0;
      end
      7'b0110011: begin // R-type
        RegWrite = 1'b1;
        ImmSrc = 2'bxx;
        ALUSrc = 1'b0;
        MemWrite = 1'b0;
        ResultSrc = 2'b00;
        Branch = 1'b0;
        ALUOp_internal = 2'b10;
        Jump = 1'b0;
      end
      7'b1100011: begin // beq
        RegWrite = 1'b0;
        ImmSrc = 2'b10;
        ALUSrc = 1'b0;
        MemWrite = 1'b0;
        ResultSrc = 2'bxx;
        Branch = 1'b1;
        ALUOp_internal = 2'b01;
```

```
        Jump = 1'b0;
      end
      7'b0010011: begin // I-type ALU
        RegWrite = 1'b1;
        ImmSrc = 2'b00;
        ALUSrc = 1'b1;
        MemWrite = 1'b0;
        ResultSrc = 2'b00;
        Branch = 1'b0;
        ALUOp_internal = 2'b10;
        Jump = 1'b0;
      end
      7'b1101111: begin // jal
        RegWrite = 1'b1;
        ImmSrc = 2'b11;
        ALUSrc = 1'bx;
        MemWrite = 1'b0;
        ResultSrc = 2'b10;
        Branch = 1'b0;
        ALUOp_internal = 2'bxx;
        Jump = 1'b1;
      end
      7'b0110111: begin // lui
        RegWrite = 1'b1;
        ImmSrc = 2'b11;
        ALUSrc = 1'bx;
        MemWrite = 1'b0;
        ResultSrc = 2'b11;
        Branch = 1'b0;
        ALUOp_internal = 2'bxx;
        Jump = 1'b0;
      end
      default: begin
        RegWrite = 1'b0;
        ImmSrc = 2'b00;
        ALUSrc = 1'b0;
        MemWrite = 1'b0;
        ResultSrc = 2'b00;
        Branch = 1'b0;
        ALUOp_internal = 2'b00;
        Jump = 1'b0;
      end
    endcase

  // ALU Decoder
  // ALU Decoder Truth Table
  // ALUOp[1:0] | funct3[2:0] | {op[5], funct7[5]} | ALUControl[2:0] | Operation
  // 00         | x           | x                   | 000             | Add
  // 01         | x           | x                   | 001             | Subtract
  // 10         | 000         | 00, 01, 10          | 000             | Add
  // 10         | 000         | 11                  | 001             | Subtract
  // 10         | 010         | x                   | 101             | SLT
  // 10         | 110         | x                   | 011             | OR
  // 10         | 111         | x                   | 010             | AND

  assign RtypeSub = funct7b5 & op[5]; // TRUE for R-type subtract instruction

  always_comb
    case(ALUOp_internal)
      2'b00: ALUControl = 3'b000; // addition
      2'b01: ALUControl = 3'b001; // subtraction
      2'b10: case(funct3) // R-type or I-type ALU
               3'b000: if (RtypeSub)
                 ALUControl = 3'b001; // subtract (funct7b5=1)
```

```
                else
                  ALUControl = 3'b000; // add (funct7b5=0)
                3'b010: ALUControl = 3'b101; // slt
                3'b110: ALUControl = 3'b011; // or
                3'b111: ALUControl = 3'b010; // and
                default: ALUControl = 3'bxxx;
              endcase
      default: ALUControl = 3'bxxx;
    endcase
endmodule

module alu(input  logic [31:0] a, b,
           input  logic [2:0]  alucontrol,
           output logic [31:0] result,
           output logic        zero);

  logic [31:0] condinvb, sum;
  logic        v;              // overflow
  logic        isAddSub;       // true when is add or subtract operation

  assign condinvb = alucontrol[0] ? ~b : b;
  assign sum = a + condinvb + alucontrol[0];
  assign isAddSub = ~alucontrol[2] & ~alucontrol[1] |
                    ~alucontrol[1] & alucontrol[0];

  always_comb
    case (alucontrol)
      3'b000:  result = sum;         // add
      3'b001:  result = sum;         // subtract
      3'b010:  result = a & b;       // and
      3'b011:  result = a | b;       // or
      3'b100:  result = a ^ b;       // xor
      3'b101:  result = sum[31] ^ v; // slt
      3'b110:  result = a << b[4:0]; // sll
      3'b111:  result = a >> b[4:0]; // srl
      default: result = 32'bx;
    endcase

  assign zero = (result == 32'b0);
  assign v = ~(alucontrol[0] ^ a[31] ^ b[31]) & (a[31] ^ sum[31]) & isAddSub;
endmodule


module datapath(input  logic        clk, reset,
                input  logic [31:0] InstrF,
                output logic [31:0] PCF,
                output logic [31:0] ALUResultM,
                output logic [31:0] WriteDataM,
                input  logic [31:0] ReadDataM,
                output logic        MemWrite);

  // Fetch stage signals
  logic [31:0] PCPlus4F, PCFa, PCTargetE;
  logic        StallF, StallD, FlushD, FlushE;
  logic        PCSrcE;

  // Decode stage signals
  logic [31:0] InstrD, PCD, PCPlus4D;
  logic [31:0] RD1D, RD2D, ImmExtD;
  logic [4:0]  Rs1D, Rs2D, RdD;
  logic        RegWriteD, MemWriteD, JumpD, BranchD;
  logic [1:0]  ResultSrcD, ImmSrcD;
  logic [2:0]  ALUControlD;
  logic        ALUSrcD;
```

```
    logic [1:0]  ALUOpD;

    // Control signals from controller
    logic [6:0]  OpD;
    logic [2:0]  Funct3D;
    logic        Funct7b5D;

    // Execute stage signals
    logic [31:0] RD1E, RD2E, PCE, ImmExtE, PCPlus4E;
    logic [31:0] SrcAE, SrcBE, WriteDataE, ALUResultE;
    logic [31:0] SrcAE_Fwd, SrcBE_Fwd;  // Intermediate signals after forwarding
    logic [4:0]  Rs1E, Rs2E, RdE;
    logic        RegWriteE, MemWriteE, JumpE, BranchE, ZeroE;
    logic [1:0]  ResultSrcE, ForwardAE, ForwardBE;
    logic [2:0]  ALUControlE;
    logic        ALUSrcE;
    logic [31:0] SL12E;

    // Memory stage signals
    logic [31:0] PCPlus4M, ReadDataMReg, SL12M;
    logic [4:0]  RdM;
    logic        RegWriteM;
    logic [1:0]  ResultSrcM;

    // Writeback stage signals
    logic [31:0] ALUResultW, ReadDataW, PCPlus4W, ResultW, SL12W;
    logic [4:0]  RdW;
    logic        RegWriteW;
    logic [1:0]  ResultSrcW;

    // Hazard detection signals
    logic [1:0]  ForwardAE_sig, ForwardBE_sig;

    // ========== FETCH STAGE ==========
    // PC mux and register
    mux2 #(32) pcmux(PCPlus4F, PCTargetE, PCSrcE, PCFa);
    flopenr #(32) pcreg(clk, reset, ~StallF, PCFa, PCF);
    adder pcadd4(PCF, 32'd4, PCPlus4F);

    // Instruction comes from memory (input port InstrF)

    // ========== FETCH/DECODE PIPELINE REGISTER ==========
    flopenrc #(32) instrdreg(clk, reset, FlushD, ~StallD, InstrF, InstrD);
    flopenrc #(32) pcdreg(clk, reset, FlushD, ~StallD, PCF, PCD);
    flopenrc #(32) pcplus4dreg(clk, reset, FlushD, ~StallD, PCPlus4F, PCPlus4D);

    // ========== DECODE STAGE ==========
    // Extract register addresses
    assign Rs1D = InstrD[19:15];
    assign Rs2D = InstrD[24:20];
    assign RdD = InstrD[11:7];
    assign OpD = InstrD[6:0];
    assign Funct3D = InstrD[14:12];
    assign Funct7b5D = InstrD[30];

    // Instantiate controller in Decode stage
    controller ctrl(OpD, Funct3D, Funct7b5D,
                    RegWriteD, ResultSrcD, MemWriteD, JumpD,
                    BranchD, ALUControlD, ALUSrcD, ImmSrcD);

    // Register file
    regfile rf(clk, RegWriteW, Rs1D, Rs2D, RdW, ResultW, RD1D, RD2D);

    // Extend immediate
```

```
    extend ext(InstrD[31:7], ImmSrcD, ImmExtD);

    // Shift left 12 for LUI
    logic [31:0] SL12D;
    assign SL12D = {InstrD[31:12], 12'b0};

    // ========== DECODE/EXECUTE PIPELINE REGISTER ==========
    floprc #(32) rd1ereg(clk, reset, FlushE, RD1D, RD1E);
    floprc #(32) rd2ereg(clk, reset, FlushE, RD2D, RD2E);
    floprc #(32) pcereg(clk, reset, FlushE, PCD, PCE);
    floprc #(32) immextereg(clk, reset, FlushE, ImmExtD, ImmExtE);
    floprc #(32) pcplus4ereg(clk, reset, FlushE, PCPlus4D, PCPlus4E);
    floprc #(5) rs1ereg(clk, reset, FlushE, Rs1D, Rs1E);
    floprc #(5) rs2ereg(clk, reset, FlushE, Rs2D, Rs2E);
    floprc #(5) rdereg(clk, reset, FlushE, RdD, RdE);

    // Control signal pipeline registers D->E
    floprc #(1) regwriteereg(clk, reset, FlushE, RegWriteD, RegWriteE);
    floprc #(2) resultsrcereg(clk, reset, FlushE, ResultSrcD, ResultSrcE);
    floprc #(1) memwriteereg(clk, reset, FlushE, MemWriteD, MemWriteE);
    floprc #(3) alucontrolereg(clk, reset, FlushE, ALUControlD, ALUControlE);
    floprc #(1) alusrcerreg(clk, reset, FlushE, ALUSrcD, ALUSrcE);
    floprc #(1) jumpereg(clk, reset, FlushE, JumpD, JumpE);
    floprc #(1) branchereg(clk, reset, FlushE, BranchD, BranchE);
    floprc #(32) sl12ereg(clk, reset, FlushE, SL12D, SL12E);

    // ========== EXECUTE STAGE ==========
    // Forwarding muxes
    mux3 #(32) forwardaemux(RD1E, ResultW, ALUResultM, ForwardAE, SrcAE_Fwd);
    mux3 #(32) forwardbemux(RD2E, ResultW, ALUResultM, ForwardBE, SrcBE_Fwd);

    // ALU source muxes
    assign SrcAE = SrcAE_Fwd;  // ALUSrcE is don't care for jal/lui
    mux2 #(32) srcbmux(SrcBE_Fwd, ImmExtE, ALUSrcE, SrcBE);
    assign WriteDataE = SrcBE_Fwd;  // WriteData is the forwarded value before immediate
selection

    // ALU
    alu alu(SrcAE, SrcBE, ALUControlE, ALUResultE, ZeroE);

    // PC target calculation
    adder pcaddbranch(PCE, ImmExtE, PCTargetE);

    // Branch decision
    assign PCSrcE = (BranchE & ZeroE) | JumpE;

    // ========== EXECUTE/MEMORY PIPELINE REGISTER ==========
    flopr #(32) aluresultmreg(clk, reset, ALUResultE, ALUResultM);
    flopr #(32) writedatamreg(clk, reset, WriteDataE, WriteDataM);
    flopr #(32) pcplus4mreg(clk, reset, PCPlus4E, PCPlus4M);
    flopr #(5) rdmreg(clk, reset, RdE, RdM);
    flopr #(1) regwritemreg(clk, reset, RegWriteE, RegWriteM);
    flopr #(1) memwritemreg(clk, reset, MemWriteE, MemWrite);
    flopr #(2) resultsrcmreg(clk, reset, ResultSrcE, ResultSrcM);
    flopr #(32) sl12mreg(clk, reset, SL12E, SL12M);

    // ========== MEMORY STAGE ==========
    // Memory read/write handled externally
    // Data memory output connected to ReadDataM input

    // ========== MEMORY/WRITEBACK PIPELINE REGISTER ==========
    flopr #(32) aluresultwreg(clk, reset, ALUResultM, ALUResultW);
    flopr #(32) readdatawreg(clk, reset, ReadDataM, ReadDataW);
    flopr #(32) pcplus4wreg(clk, reset, PCPlus4M, PCPlus4W);
```

```
  flopr #(5) rdwreg(clk, reset, RdM, RdW);
  flopr #(1) regwritewreg(clk, reset, RegWriteM, RegWriteW);
  flopr #(2) resultsrcwreg(clk, reset, ResultSrcM, ResultSrcW);
  flopr #(32) sl12wreg(clk, reset, SL12M, SL12W);

  // ========== WRITEBACK STAGE ==========
  // Result mux
  mux4 #(32) resultmux(ALUResultW, ReadDataW, PCPlus4W, SL12W, ResultSrcW, ResultW);

  // ========== HAZARD UNIT ==========
  hazard hu(Rs1D, Rs2D, Rs1E, Rs2E, RdE, RdM, RdW,
            PCSrcE, ResultSrcE, RegWriteM, RegWriteW,
            ForwardAE, ForwardBE, StallF, StallD, FlushD, FlushE);

endmodule

module regfile(input  logic        clk,
               input  logic        we3,
               input  logic [ 4:0] a1, a2, a3,
               input  logic [31:0] wd3,
               output logic [31:0] rd1, rd2);

  logic [31:0] rf[31:0];

  // three ported register file
  // read two ports combinationally (A1/RD1, A2/RD2)
  // write third port on rising edge of clock (A3/WD3/WE3)
  // register 0 hardwired to 0

  always_ff @(posedge clk)
    if (we3) rf[a3] <= wd3;

  assign rd1 = (a1 != 0) ? rf[a1] : 0;
  assign rd2 = (a2 != 0) ? rf[a2] : 0;
endmodule

module adder(input  [31:0] a, b,
             output [31:0] y);

  assign y = a + b;
endmodule

module extend(input  logic [31:7] instr,
              input  logic [1:0]  immsrc,
              output logic [31:0] immext);

  always_comb
    case(immsrc)
               // I-type
      2'b00:   immext = {{20{instr[31]}}, instr[31:20]};
               // S-type (stores)
      2'b01:   immext = {{20{instr[31]}}, instr[31:25], instr[11:7]};
               // B-type (branches)
      2'b10:   immext = {{20{instr[31]}}, instr[7], instr[30:25], instr[11:8], 1'b0};
               // J-type (jal)
      2'b11:   immext = {{12{instr[31]}}, instr[19:12], instr[20], instr[30:21], 1'b0};
      default: immext = 32'bx; // undefined
    endcase
endmodule

module hazard(input  logic [4:0] Rs1D, Rs2D, Rs1E, Rs2E,
              input  logic [4:0] RdE, RdM, RdW,
              input  logic       PCSrcE,
              input  logic [1:0] ResultSrcE,
```

```
               input  logic       RegWriteM, RegWriteW,
               output logic [1:0] ForwardAE, ForwardBE,
               output logic       StallF, StallD, FlushD, FlushE);

  logic lwStall;

  // Forwarding logic
  always_comb begin
    // Default: no forwarding
    ForwardAE = 2'b00;
    ForwardBE = 2'b00;

    // ForwardAE logic
    if (RegWriteM && (RdM != 0) && (RdM == Rs1E))
      ForwardAE = 2'b10; // Forward from Memory stage
    else if (RegWriteW && (RdW != 0) && (RdW == Rs1E))
      ForwardAE = 2'b01; // Forward from Writeback stage

    // ForwardBE logic
    if (RegWriteM && (RdM != 0) && (RdM == Rs2E))
      ForwardBE = 2'b10; // Forward from Memory stage
    else if (RegWriteW && (RdW != 0) && (RdW == Rs2E))
      ForwardBE = 2'b01; // Forward from Writeback stage
  end

  // Stall logic for load-use hazard
  assign lwStall = ResultSrcE[0] && ((RdE == Rs1D) || (RdE == Rs2D));
  assign StallF = lwStall;
  assign StallD = lwStall;

  // Flush logic
  assign FlushD = PCSrcE;  // Flush decode stage on branch/jump
  assign FlushE = lwStall | PCSrcE;  // Flush execute on stall or branch/jump

endmodule

module flopr #(parameter WIDTH = 8)
              (input  logic             clk, reset,
               input  logic [WIDTH-1:0] d,
               output logic [WIDTH-1:0] q);

  always_ff @(posedge clk, posedge reset)
    if (reset) q <= 0;
    else       q <= d;
endmodule

module flopenr #(parameter WIDTH = 8)
               (input  logic             clk, reset, en,
                input  logic [WIDTH-1:0] d,
                output logic [WIDTH-1:0] q);

  always_ff @(posedge clk, posedge reset)
    if (reset)   q <= 0;
    else if (en) q <= d;
endmodule

module floprc #(parameter WIDTH = 8)
              (input  logic             clk, reset, clear,
               input  logic [WIDTH-1:0] d,
               output logic [WIDTH-1:0] q);

  always_ff @(posedge clk, posedge reset)
    if (reset)      q <= 0;
    else if (clear) q <= 0;
```

```
    else            q <= d;
endmodule

module flopenrc #(parameter WIDTH = 8)
                (input  logic             clk, reset, clear, en,
                 input  logic [WIDTH-1:0] d,
                 output logic [WIDTH-1:0] q);

  always_ff @(posedge clk, posedge reset)
    if (reset)      q <= 0;
    else if (clear) q <= 0;
    else if (en)    q <= d;
endmodule

module mux2 #(parameter WIDTH = 8)
             (input  logic [WIDTH-1:0] d0, d1,
              input  logic             s,
              output logic [WIDTH-1:0] y);

  assign y = s ? d1 : d0;
endmodule

module mux3 #(parameter WIDTH = 8)
             (input  logic [WIDTH-1:0] d0, d1, d2,
              input  logic [1:0]       s,
              output logic [WIDTH-1:0] y);

  assign y = s[1] ? d2 : (s[0] ? d1 : d0);
endmodule

module mux4 #(parameter WIDTH = 8)
             (input  logic [WIDTH-1:0] d0, d1, d2, d3,
              input  logic [1:0]       s,
              output logic [WIDTH-1:0] y);

  assign y = s[1] ? (s[0] ? d3 : d2) : (s[0] ? d1 : d0);
endmodule

module imem(input  logic [31:0] a,
            output logic [31:0] rd);

  logic [31:0] RAM[63:0];

  initial
    $readmemh("/mnt/shared/sangeeth/Documents/RPI/Semester7/ECSE-
4770/Labs/Lab9/riscvtest.txt",RAM);

  assign rd = RAM[a[31:2]]; // word aligned
endmodule

module dmem(input  logic        clk, we,
            input  logic [31:0] a, wd,
            output logic [31:0] rd);

  logic [31:0] RAM[63:0];

  assign rd = RAM[a[31:2]]; // word aligned

  always_ff @(posedge clk)
    if (we) RAM[a[31:2]] <= wd;
endmodule
```